

RESEARCH ARTICLE | AUGUST 28 2023

MLIP-3: Active learning on atomic environments with moment tensor potentials

Special Collection: [Software for Atomistic Machine Learning](#)

Evgeny Podryabinkin ; Kamil Garifullin ; Alexander Shapeev ; Ivan Novikov  



J. Chem. Phys. 159, 084112 (2023)

<https://doi.org/10.1063/5.0155887>



Articles You May Be Interested In

Moment tensor potential and equivariant tensor network potential with explicit dispersion interactions

J. Chem. Phys. (October 2025)

Low-rank matrix and tensor approximations for compression of machine-learning interatomic potentials

J. Chem. Phys. (December 2025)

Maximizing efficiency of dataset compression for machine learning potentials with information theory

J. Chem. Phys. (April 2026)

13 May 2026 08:07:04



Freedom to Innovate.

The New VHFLI 200 MHz Lock-in Amplifier.

Orchestrate pulses, triggers, and acquisition as the hub of your experiment. Discover more – run every signal analysis tool, simultaneously.

Order now

MLIP-3: Active learning on atomic environments with moment tensor potentials

Cite as: J. Chem. Phys. 159, 084112 (2023); doi: 10.1063/5.0155887

Submitted: 24 April 2023 • Accepted: 28 July 2023 •

Published Online: 28 August 2023



Evgeny Podryabinkin,¹  Kamil Garifullin,²  Alexander Shapeev,¹  and Ivan Novikov^{1,2,a)} 

AFFILIATIONS

¹Skolkovo Institute of Science and Technology, Skolkovo Innovation Center, Bolshoy boulevard 30, Moscow 143026, Russian Federation

²Moscow Institute of Physics and Technology, Institutsky per., 9, Dolgoprudny, Moscow region, 141700 Russian Federation

Note: This paper is part of the JCP Special Topic on Software for Atomistic Machine Learning.

a) Author to whom correspondence should be addressed: i.novikov@skoltech.ru

ABSTRACT

Nowadays, academic research relies not only on sharing with the academic community the scientific results obtained by research groups while studying certain phenomena but also on sharing computer codes developed within the community. In the field of atomistic modeling, these were software packages for classical atomistic modeling, and later for quantum-mechanical modeling; currently, with the fast growth of the field of machine-learning potentials, the packages implement such potentials. In this paper, we present the MLIP-3 package for constructing moment tensor potentials and performing their active training. This package builds on the MLIP-2 package [Novikov *et al.*, “The MLIP package: moment tensor potentials with MPI and active learning,” *Mach. Learn.: Sci. Technol.*, **2**(2), 025002 (2020)], however, with a number of improvements, including active learning on atomic neighborhoods of a possibly large atomistic simulation.

Published under an exclusive license by AIP Publishing. <https://doi.org/10.1063/5.0155887>

I. INTRODUCTION

The success of machine-learning potentials in recent years is revolutionizing computational materials science. Due to their unique combination of accuracy and performance, they have enabled solving many atomistic modeling problems that were previously impossible or impractical to solve due to excessive consumption of computational resources.^{1–5} Machine-learning potentials are based on the idea of training a classical (as opposed to quantum-mechanical) interatomic interaction model on the results of first-principles calculations and further reproducing the behavior of this model in practical simulations. The feature that distinguishes machine-learning potentials from semiempirical-based models with coefficients found from first-principles calculations is their flexible functional form that allows the potential energy surface to be approximated with arbitrary accuracy by increasing the number of parameters and consequently increasing the computation cost, however, keeping it orders of magnitude below the cost of first-principles calculations.

A number of machine-learning potentials are being developed, including SNAP—the potentials on the basis of spherical

harmonics,⁶ GAP—the potentials based on Gaussian processes,⁷ ANN—the potentials based on artificial neural networks,⁸ and moment tensor potentials (MTP)—the polynomial-like potentials based on the tensors of inertia of atomistic environments.⁹ Examples of more recent potentials are polynomial-like ACE potentials^{10,11} and neural network-based DeepMD,¹² PINN,¹³ and NeQUIP.¹⁴

In 2020,¹⁵ an independent benchmarking of the first four potentials was carried out, in which MTPs (moment tensor potentials) showed a very favorable performance in terms of accuracy vs computational efficiency. The MTPs and tools to work with them have been implemented and made available for use in the software package MLIP-2.¹⁶ The MTP potentials have been used in solving a number of materials science problems including predicting crystal structures,¹⁷ searching for new materials and high-performance screening of structures,¹ modeling chemical reactions,^{2,18} calculating melting points,¹⁶ thermal conductivity,^{3,19} hardness,⁴ diffusion coefficients,⁵ and other computationally expensive properties of materials.²⁰

Three years have passed since MLIP-2 code was published, and during that time, numerous changes and improvements have been implemented in the new version of the code. These changes are

related to the simplification of operations with machine-learning potentials, including their active learning and new capabilities of parallel calculations. Besides that, we present a new feature to train MTPs on local fragments of a large atomistic configuration in the new software version. The purpose of this article was to introduce the reader to the MLIP-3 package and describe its features. It should be noted that the functional form of MTPs, as well as their learning and extrapolation detection algorithms, remained essentially unchanged compared with the MLIP-2 package.¹⁶ At the same time, the user interface underwent substantial revision, new parallelism capabilities were added, and perhaps most importantly, the possibility of active learning on atomistic environments (neighborhoods) was implemented.

II. THEORY

A. Moment tensor potential

Moment tensor potential (MTP) is a machine-learning interatomic potential that was first proposed in Ref. 9 for single-component materials, and its generalizations were described in Refs. 21 and 22. MTP is a local potential, i.e., the energy E^{MTP} is the sum of contributions $V^{\text{MTP}}(\mathbf{n}_i)$ of atomic neighborhoods $\mathbf{n}_i$ for N atoms,

$$E^{\text{MTP}} = \sum_{i=1}^N V^{\text{MTP}}(\mathbf{n}_i). \quad (1)$$

Each neighborhood is a tuple $\mathbf{n}_i = (\{r_{i1}, z_{i1}, z_{i1}\}, \dots, \{r_{ij}, z_{ij}, z_{ij}\}, \dots, \{r_{iN_{\text{nbh}}}, z_{iN_{\text{nbh}}}, z_{iN_{\text{nbh}}}\})$, where r_{ij} are relative atomic positions, z_i and z_j are the types of central and neighboring atoms, and N_{nbh} is the number of atoms in neighborhood. We also denote the maximum number of atomic types occurred in all the neighborhoods by N_{types} . Each contribution $V^{\text{MTP}}(\mathbf{n}_i)$ in the potential energy E^{MTP} expands through a set of basis functions,

$$V^{\text{MTP}}(\mathbf{n}_i) = \sum_{\alpha=1}^{N_{\text{lin}}} \xi_{\alpha} B_{\alpha}(\mathbf{n}_i), \quad (2)$$

where B_{α} are the MTP basis functions, ξ_{α} are the linear parameters to be found, and N_{lin} is the number of these parameters. To define the functional form of the MTP basis functions and the number N_{lin} , we introduce the so-called moment tensor descriptors,

$$M_{\mu,v}(\mathbf{n}_i) = \sum_{j=1}^{N_{\text{nbh}}} f_{\mu}(|r_{ij}|, z_i, z_j) r_{ij}^{\otimes v}. \quad (3)$$

The descriptor consists of the angular part $r_{ij}^{\otimes v}$ (the symbol “ $\otimes$ ” denotes the outer product of vectors, and thus, the angular part is the tensor of v -th order) and the radial part $f_{\mu}(|r_{ij}|, z_i, z_j)$ of the following form:

$$f_{\mu}(|r_{ij}|, z_i, z_j) = \sum_{\beta=1}^{N_{\text{polyn}}} c_{\mu,z_i,z_j}^{(\beta)} T^{(\beta)}(|r_{ij}|) (R_{\text{cut}} - |r_{ij}|)^2. \quad (4)$$

Here, $\mu = 0, \dots, N_{\text{rad}} - 1$ is the number of the radial function f_{μ} (the method to define a concrete number N_{rad} of radial functions needed to construct all the MTP basis functions is detailed below), $c_{\mu,z_i,z_j}^{(\beta)}$ are the radial parameters to be found, $T^{(\beta)}(|r_{ij}|)$ are polynomial functions, and N_{polyn} is their number. The number of the

radial MTP parameters $c_{\mu,z_i,z_j}^{(\beta)}$ is $N_{\text{polyn}} \times N_{\text{rad}} \times N_{\text{types}}^2$. Finally, the term $(R_{\text{cut}} - |r_{ij}|)^2$ is introduced to ensure smoothness with respect to the atoms leaving and entering the sphere with the cutoff radius R_{cut} .

By definition, the MTP basis function B_{α} is a contraction of one or more moment tensor descriptors, yielding a scalar, e.g.,

$$M_{1,0}, (M_{1,2} M_{0,1}) \cdot M_{0,1}, M_{1,2} : M_{3,2}, \dots,$$

where “ $\cdot$ ” is an inner product of vectors, and “ $:$ ” is a Frobenius product of matrices. In order to construct the basis functions B_{α} and, thus, determine a particular functional form of MTP, we define the so-called *level* of moment tensor descriptor,

$$\text{lev} M_{\mu,v} = 2 + 4\mu + v, \quad (5)$$

for example, $\text{lev} M_{0,0} = 2$, $\text{lev} M_{2,1} = 11$, and $\text{lev} M_{0,3} = 5$. We also define the level of MTP basis function,

$$\text{lev} B_{\alpha} = \text{lev} \underbrace{\prod_{p=1}^P M_{\mu_p, v_p}}_{\text{scalar}} = \sum_{p=1}^P (2 + 4\mu_p + v_p). \quad (6)$$

A set of MTP basis functions and, thus, a particular functional form of MTP depends on the maximum level, lev_{max} , which we also call the level of MTP. We include in the set of the MTP basis functions only the ones with $\text{lev} B_{\alpha} \leq \text{lev}_{\text{max}}$, e.g., the MTP of the sixth level includes five basis functions:

$$B_1 = M_{0,0}; \text{lev} B_1 = 2 \leq \text{lev}_{\text{max}} = 6,$$

$$B_2 = M_{1,0}; \text{lev} B_2 = 6 \leq \text{lev}_{\text{max}} = 6,$$

$$B_3 = M_{0,0}^2; \text{lev} B_3 = 4 \leq \text{lev}_{\text{max}} = 6,$$

$$B_4 = M_{0,1} \cdot M_{0,1}; \text{lev} B_4 = 6 \leq \text{lev}_{\text{max}} = 6,$$

$$B_5 = M_{0,0}^3; \text{lev} B_5 = 6 \leq \text{lev}_{\text{max}} = 6.$$

Thus, the number of linear parameters N_{lin} and the number N_{rad} of radial functions depend on the level of MTP and are fixed for each level (e.g., if $\text{lev}_{\text{max}} = 6$ then $N_{\text{lin}} = 5$, $N_{\text{rad}} = 2$).

B. Fitting

For finding MTP parameters, we fit them on a (quantum-mechanical) training set. Let K be the number of configurations in the training set, and $N^{(k)}$ is the number of atoms in k -th configuration. Denote a vector of MTP parameters to be found by $\theta = (\xi_{\alpha}, c_{\mu,z_i,z_j}^{(\beta)})$. We find the optimal parameters $\bar{\theta}$ by solving the following optimization problem (minimization of the objective function),

$$\sum_{k=1}^K \left[w_e (E_k^{\text{MTP}}(\theta) - E_k^{\text{QM}})^2 + w_t \sum_{i=1}^{N^{(k)}} |\mathbf{f}_{i,k}^{\text{MTP}}(\theta) - \mathbf{f}_{i,k}^{\text{QM}}|^2 + w_s \sum_{a,b=1}^3 (\sigma_{ab,k}^{\text{MTP}}(\theta) - \sigma_{ab,k}^{\text{QM}})^2 \right] \rightarrow \min. \quad (7)$$

We start with randomly initialized MTP parameters. The optimal parameters θ are found numerically by using the iterative method for minimization of the nonlinear objective function, namely, Broyden–Fletcher–Goldfarb–Shanno algorithm.²³ Thus, after the optimization, the parameters θ are near the local minimum of the objective function. In this objective function, E_k^{QM} , $f_{i,k}^{\text{QM}}$, and $\sigma_{ab,k}^{\text{QM}}$ are the reference energies, forces, and stresses, i.e., the ones to which we fit the MTP energies E_k^{MTP} , forces $f_{i,k}^{\text{MTP}}$, and stresses $\sigma_{ab,k}^{\text{MTP}}$ and, thus, optimize the MTP parameters θ . The factors w_e , w_f , and w_s are non-negative weights that express the importance of energies, forces, and stresses with respect to each other. We refer to the minimization problem (7) as the fitting of MTP.

C. Active learning: Query strategy on atomic configurations

We introduce a concept of *extrapolation grade* on the basis of the papers.^{21,24} To that end, we start from (7). Assume that we know the vector of optimal MTP parameters $\bar{\theta}$ and let the length of the vector be m . Then, we can linearize each term in the objective function, in particular, the term with the difference between energies,

$$E_k^{\text{QM}} - E_k^{\text{MTP}}(\theta) \approx E_k^{\text{QM}} - \sum_{p=1}^m (\theta_p - \bar{\theta}_p) \frac{\partial E_k^{\text{MTP}}(\bar{\theta})}{\partial \theta_p}.$$

We can consider the fitting of MTP as the solution of the overdetermined system,

$$\sum_{p=1}^m \theta_p \frac{\partial E_k^{\text{MTP}}(\bar{\theta})}{\partial \theta_p} = E_k^{\text{QM}} + \sum_{p=1}^m \bar{\theta}_p \frac{\partial E_k^{\text{MTP}}(\bar{\theta})}{\partial \theta_p}. \quad (8)$$

The matrix of system (8) is

$$B = \begin{pmatrix} \frac{\partial E_1^{\text{MTP}}}{\partial \theta_1}(\bar{\theta}) & \dots & \frac{\partial E_1^{\text{MTP}}}{\partial \theta_m}(\bar{\theta}) \\ \vdots & & \vdots \\ \frac{\partial E_K^{\text{MTP}}}{\partial \theta_1}(\bar{\theta}) & \dots & \frac{\partial E_K^{\text{MTP}}}{\partial \theta_m}(\bar{\theta}) \end{pmatrix}.$$

From this set of equations, we select a set of the m most linearly independent equations, that is, the equations with the maximum absolute value of the determinant. This is the core idea of our active learning strategy, which is based on the D-optimality criterion. According to this criterion, the configurations that give the most linearly independent equations (maximizing the absolute value of the determinant) are selected and included in the training set. We call the selected set of m configurations the *active set*, which is the set of the most “extreme” and diverse configurations from the point of view of MTP that is being fitted.

To construct an active set from a set of configurations, we use the MaxVol algorithm,²⁵ providing a computationally efficient way to calculate how many times the absolute value of the determinant of a square matrix A associated with the active set will change, if some configuration from the active set is replaced by some another. Let

$$b = \left(\frac{\partial E^{\text{MTP}}}{\partial \theta_1}(\bar{\theta}) \dots \frac{\partial E^{\text{MTP}}}{\partial \theta_m}(\bar{\theta}) \right),$$

the coefficients in Eq. (8) calculated for a candidate configuration cfg . Then, the largest increase in $|\det(A)|$ is

$$\gamma(\text{cfg}) = \max_{1 \leq j \leq m} |c_j|,$$

where

$$(c_1 \dots c_m) = \left(\frac{\partial E^{\text{MTP}}}{\partial \theta_1}(\bar{\theta}) \dots \frac{\partial E^{\text{MTP}}}{\partial \theta_m}(\bar{\theta}) \right) A^{-1}.$$

Thus, if $\gamma(\text{cfg}) > 1$, then $|\det(A)|$ could be increased γ times when a configuration from the active set associated with index $j_0 = \text{argmax}_j(c_j)$ is replaced by cfg .

Note that if the active set and the training set are the same, then the MTP parameters can be found as

$$\xi = (E^{\text{qm}}(\text{cfg}_1) \dots E^{\text{qm}}(\text{cfg}_m)) A^{-1},$$

and the energy of any configuration can be expressed as a linear combination of energies of active set configurations,

$$E^{\text{mtp}}(\text{cfg}) = \sum_{j=1}^m c_j E^{\text{qm}}(\text{cfg}_j),$$

and $\gamma(\text{cfg}) = \max_j |c_j|$. Therefore, we say that MTP extrapolates if $\gamma(\text{cfg}) > 1$ on cfg and interpolates otherwise. Hence, γ can be interpreted as the *extrapolation grade*.

D. Active learning: Query strategy on local atomic neighborhoods

In Sec. II C, we formulated the strategy of active learning on atomistic configurations. This strategy works fine if we deal with relatively small configurations, which can be calculated with quantum-mechanical methods spending reasonable computational resources. However, this becomes impractical when we deal with relatively large configurations because their quantum-mechanical calculations are too computationally expensive. Moreover, MTP may, in some sense, extrapolate only on some small part of the large configuration, whereas the rest of the configuration can be described well by the existing training set. In this case, it may be desirable to use the active learning strategy in local atomic neighborhoods.

Before we formulate this strategy, we note that the active learning on atomistic configurations does not use the information in the right part of (8) (i.e., does not use the quantum-mechanical energies). Thus, if we assume that we are able to somehow calculate individual atomic energy $V^{\text{QM}}(\mathbf{n}_i^{(k)})$, then we can write the overdetermined system of equations for each atomic neighborhood in every configuration from the training set,

$$V^{\text{MTP}}(\mathbf{n}_i^{(k)}) = V^{\text{QM}}(\mathbf{n}_i^{(k)}), 1 \leq i \leq N^{(k)}, 1 \leq k \leq K. \quad (9)$$

After linearization with respect to the parameters θ near the optimal values $\bar{\theta}$, we obtain

$$\sum_{p=1}^m \theta_p \frac{\partial V^{\text{MTP}}(\bar{\theta}, \mathbf{n}_i^{(k)})}{\partial \theta_p} = V_{i,k}^{\text{QM}} + \sum_{p=1}^m \bar{\theta}_p \frac{\partial V^{\text{MTP}}(\bar{\theta}, \mathbf{n}_i^{(k)})}{\partial \theta_p}, 1 \leq k \leq K. \quad (10)$$

The matrix of this system (10) is

$$B = \begin{pmatrix} \frac{\partial V^{\text{MTP}}}{\partial \theta_1}(\tilde{\theta}, \mathbf{n}_1^{(1)}) & \dots & \frac{\partial V^{\text{MTP}}}{\partial \theta_m}(\tilde{\theta}, \mathbf{n}_1^{(1)}) \\ \vdots & & \vdots \\ \frac{\partial V^{\text{MTP}}}{\partial \theta_1}(\tilde{\theta}, \mathbf{n}_{N^{(1)}}^{(1)}) & \dots & \frac{\partial V^{\text{MTP}}}{\partial \theta_m}(\tilde{\theta}, \mathbf{n}_{N^{(1)}}^{(1)}) \\ \vdots & & \vdots \\ \frac{\partial V^{\text{MTP}}}{\partial \theta_1}(\tilde{\theta}, \mathbf{n}_1^{(K)}) & \dots & \frac{\partial V^{\text{MTP}}}{\partial \theta_m}(\tilde{\theta}, \mathbf{n}_1^{(K)}) \\ \vdots & & \vdots \\ \frac{\partial V^{\text{MTP}}}{\partial \theta_1}(\tilde{\theta}, \mathbf{n}_{N^{(K)}}^{(K)}) & \dots & \frac{\partial V^{\text{MTP}}}{\partial \theta_m}(\tilde{\theta}, \mathbf{n}_{N^{(K)}}^{(K)}) \end{pmatrix},$$

where the first $N^{(1)}$ rows correspond to the first configuration (of $N^{(1)}$ atoms) in the training set, and the last $N^{(K)}$ rows correspond to the last (K -th) configuration; thus, the size of the matrix is $\left(\sum_{k=1}^K N^{(k)}\right) \times m$.

The active learning strategy on atomic neighborhoods can be formulated in the same manner as active learning on configurations. The active set in this case consists of m atomistic neighborhoods, whereas the matrix of maximal volume A (i.e., square matrix with the maximal absolute value of determinant) contains the corresponding elements $\frac{\partial V^{\text{MTP}}}{\partial \theta_p}(\tilde{\theta}, \mathbf{n}_i^{(k)})$, where $\mathbf{n}_i^{(k)}$ is the neighborhood of the i -th atom in the k -th configuration.

Given a configuration cfg^* , we checked if it has an extrapolative neighborhood in the following way. We calculate the matrix

$$C = \begin{pmatrix} \frac{\partial V^{\text{MTP}}}{\partial \theta_1}(\tilde{\theta}, \mathbf{n}_1^*) & \dots & \frac{\partial V^{\text{MTP}}}{\partial \theta_p}(\tilde{\theta}, \mathbf{n}_1^*) & \dots & \frac{\partial V^{\text{MTP}}}{\partial \theta_m}(\tilde{\theta}, \mathbf{n}_1^*) \\ \vdots & & \vdots & & \vdots \\ \frac{\partial V^{\text{MTP}}}{\partial \theta_1}(\tilde{\theta}, \mathbf{n}_i^*) & \dots & \frac{\partial V^{\text{MTP}}}{\partial \theta_p}(\tilde{\theta}, \mathbf{n}_i^*) & \dots & \frac{\partial V^{\text{MTP}}}{\partial \theta_m}(\tilde{\theta}, \mathbf{n}_i^*) \\ \vdots & & \vdots & & \vdots \\ \frac{\partial V^{\text{MTP}}}{\partial \theta_1}(\tilde{\theta}, \mathbf{n}_N^*) & \dots & \frac{\partial V^{\text{MTP}}}{\partial \theta_p}(\tilde{\theta}, \mathbf{n}_N^*) & \dots & \frac{\partial V^{\text{MTP}}}{\partial \theta_m}(\tilde{\theta}, \mathbf{n}_N^*) \end{pmatrix} A^{-1}.$$

The maximal by absolute value element $\gamma_i = \gamma(\mathbf{n}_i^*) = \max_{1 \leq p \leq m} |C_{i,p}|$ in each row i is the extrapolation degree for the corresponding neighborhood $\mathbf{n}_i^*$. If $\gamma_i > 1$, then MTP will extrapolate on the corresponding neighborhood $\mathbf{n}_i^*$.

E. Active learning on-the-fly

According to our experience, the learning on-the-fly scheme described in this section is one of the most efficient methods to fit an MTP for many applications. It allows accumulating only relevant configurations in the training set that often results in optimal training errors because this MTP interpolates in the minimal required phase space. According to this scheme, we start with an MTP initially trained on some (typically minimal) amount of relevant configurations and repeat the following steps (see also Fig. 1) until the desired simulations (e.g., MD simulations with a range

of temperatures and pressures) is completed without encountering extrapolative configurations or neighborhoods (Fig. 2).

1. **Run the simulation with extrapolation control.** The simulation algorithm generates configurations, for which energy and/or forces are computed. Prior to this calculation, each configuration is assessed for extrapolation according to one of the two aforementioned active learning strategies. That is, the extrapolation grade is calculated either for the entire configuration or for each neighborhood in this configuration. In the latter case, the extrapolation degree of configuration is defined as the maximal among extrapolation grades of all neighborhoods $\gamma(\text{cfg}) = \max_i \gamma(\mathbf{n}_i)$. Depending on the degree of extrapolation $\gamma(\text{cfg})$, we can (i) either proceed with calculation of the energy and/or forces in the case of interpolation or insignificant extrapolation, $\gamma(\text{cfg}) < \gamma_{\text{save}}$; or (ii) save the configuration or its fragment to the file, and proceed with the

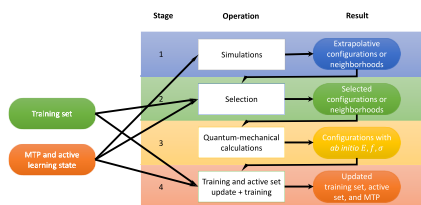


FIG. 1. Scheme of active learning bootstrapping iterations.



FIG. 2. Classification of extrapolation grades. A configuration with a grade less than γ_{save} does not trigger any active learning actions, and a configuration whose grade is between γ_{save} and γ_{break} is considered reliable yet useful for extension of the training set, but those with a grade larger than γ_{break} are risky and trigger termination of the simulation.

calculation of the energy and/or forces, if extrapolation is significant but not critically large $\gamma_{\text{save}} < \gamma(\text{cfg}) < \gamma_{\text{break}} \approx 10$; or (iii) break the calculation if extrapolation degree exceeds the critical threshold $\gamma(\text{cfg}) > \gamma_{\text{break}}$ (see Fig. 1).

2. **Selection of configurations or neighborhoods to be learned.**

The extrapolative configurations or the fragments of configurations with extrapolative environments saved to the file may be learned and improve reliability (transferability) of the potential. However, the saved files may contain a large number of similar configurations or neighborhoods. According to our active learning strategy, we select among the extrapolative configurations or neighborhoods that maximize the volume of the matrix associated with the active set.

3. **Quantum-mechanical calculations.** After selecting the appropriate configurations or neighborhoods for learning, it is necessary to calculate their quantum-mechanical energy, forces, and stresses. In the case of active learning on configurations, the selected configurations are calculated with the quantum-mechanical model. In the case of active learning in neighborhoods, there are three ways of processing the selected neighborhoods. (i) The simplest way is to directly calculate the configurations containing selected neighborhoods with the quantum-mechanical model. However, this option is typically impractical since these configurations are too large to be treated with the quantum-mechanical model. Hence, we can either (ii) cut the extrapolative neighborhood from the configurations and treat them as non-periodic clusters with the quantum-mechanical model or (iii) construct periodic structure from each extrapolative neighborhood in a manual or semi-manual manner. Option (ii) is implemented in our MLIP-3 package, whereas for option (iii), the users should write their own neighborhood processing code.
4. **MTP re-training.** After performing quantum-mechanical calculations on the training data calculated at the previous step, it is added to the training set, updates the active set, and the potential is re-trained.

F. Protocol for choosing DFT parameters for neighborhood calculations

In the case of active learning on atomic neighborhoods when using the density functional theory (DFT) code with periodic boundary conditions (e.g., VASP) for calculating atomic neighborhoods, the following steps for choosing DFT parameters should be done.

- Choose the radius of the neighborhood (the radius should be greater than the cutoff radius of the potential) and cut the neighborhood from the large configuration.
- Choose $1 \times 1 \times 1$ k-mesh (use this k-mesh for the neighborhood DFT calculations), and the energy cutoff is greater than the energy cutoff in pseudopotentials. Add vacuum along each direction and investigate the potential energy convergence with respect to vacuum size. Find the optimal vacuum size.
- Investigate the potential energy convergence with respect to the energy cutoff with the fixed optimal vacuum size along each direction. Find the optimal energy cutoff.

- Take the optimal vacuum size and the energy cutoff and investigate the convergence of the force on the central atom in the neighborhood with respect to the radius of the neighborhood. Thus, choose the radius of the neighborhood.

III. MLIP-3 PACKAGE DESCRIPTION

MLIP-3 is an open-source software package available at <https://gitlab.com/ashapeev/mlip-3>. It provides the code compiled to the executable file `mlp` implementing a number of commands manipulating with machine-learning potentials and quantum-mechanical database. Additionally, the plugin for using these potentials with the LAMMPS package²⁶ is available at

<https://gitlab.com/ivannovikov/interface-lammps-mlip-3/>

A. MLIP-3 files and commands

The atomic configurations are stored in the MLIP's native `cfg` files in a plain-text format. These files contain the size of configuration, atomic numbers, types, positions (in Å), supercell (in Å), and (optionally) energies (in eV), forces (in eV/Å), stresses (in eV), features (additional textual attributes of configuration, e.g., extrapolation grade), and `nbh_grades` (or, extrapolation grade per atom, needed for query strategy on atomic neighborhoods). The reader can find more details on `cfg` format in Ref. 27.

The training set is written in `cfg` files, i.e., the configurations obtained with the codes for quantum-mechanical calculations (e.g., DFT calculations) should be converted from the formats of these codes to the native MLIP `cfg` before fitting of MTP. For example, we convert the configurations in the VASP format to the MLIP format by executing

```
mlp convert OUTCAR out.cfg -input_format = outcar
```

where OUTCAR is the VASP output file (which is the input file for the `convert` command) with energies, forces, and stresses calculated, and `out.cfg` is the output file with the energies, forces, and stresses saved in the internal MLIP `cfg` format. The atomic types written in `out.cfg` are relative after executing the above command, i.e., they do not correspond to the numbers of the elements in the Periodic Table. The `absolute_types` option allows us to write absolute atomic types in the `cfg` file while converting from OUTCAR instead of, e.g., 0, 1, 2, ...

Templates of MTPs corresponding to different levels of MTP are stored in the MLIP's native `.almtp` files. The MLIP-3 package contains untrained potentials defining MTPs of level 6, ..., 28 in the folder `MTP_templates` of the main package folder. The templates of untrained MTPs are described in Ref. 27 in detail. To fit an MTP, we run the following command

```
mlp train in.almtp train.cfg -save_to = pot.almtp
```

The `in.almtp` file contains the initial MTP, but it does not have to contain parameters in which case we start from the

randomly initialized parameters. The `-save_to = pot.almtp` option indicates that the trained MTP will be saved to the `pot.almtp` file (if this option is skipped in `pot.almtp` file will be overwritten). This file contains the optimal MTP parameters, the query strategy (“on configurations” or “on local atomic neighborhoods”), the matrix corresponding to the active set, and the configurations from the active set. We further refer to the query strategy “on configurations” as the “configuration-based” active learning mode and the query strategy “neighborhoods-based” as the “local atomic neighborhoods” active learning mode. The default active learning mode is “configuration-based,” but it can be specified with the option `-al_mode = cfg` or `-al_mode = nbh`. Active learning mode is set up at the first training and should not be changed after.

We emphasize that after training, the `.almtp` file always contains information on the active learning mode and the active set, i.e., all the data related to active learning will be automatically updated after each re-training. However, MTPs can also be used in the passive learning mode. In this case, the data related to active learning will simply be ignored.

We check the fitting errors by executing

```
mlp check_errors pot.almtp train.cfg
```

The computed errors will be reported to the screen.

Calculation of the energies, forces, and stresses for the configurations in a file `in.cfg` is done by

```
mlp calculate_efs pot.almtp in.cfg efs.cfg
```

For estimation of extrapolation grades of configurations/per atoms, the following command is used:

```
mlp calculate_grade pot.almtp in.cfg grade.cfg
```

We note that all the data needed for these commands will be loaded from the files `pot.almtp` and `in.cfg`.

The trained MTP can be used for different types of atomistic simulations, e.g., relaxation (minimization of the potential energy) of structures. To that end, we run the command

```
mlp relax pot.almtp to_relax.cfg relaxed.cfg
```

The `relax` command can be used both without the control of extrapolation as shown above and with the control of extrapolation. The `-extrapolation_control = true` option turns on calculating the extrapolation grade for each configuration or the extrapolation grade per atom, depending on the active learning mode (configured at the first training). This option also turns on saving either extrapolative periodic configurations or non-periodic parts of the large atomic systems including the extrapolative neighborhoods for which either $\gamma_{\text{save}} < \gamma(\text{cfg}) < \gamma_{\text{break}}$ or $\gamma_{\text{save}} < \gamma(\mathbf{n}_i^*) < \gamma_{\text{break}}$. If $\gamma(\text{cfg}) > \gamma_{\text{break}}$ (or, $\gamma(\mathbf{n}_i^*) > \gamma_{\text{break}}$), then relaxation terminates.

The value of the γ_{save} threshold can be specified with the option `-threshold_save = (value)`, the value of the γ_{break} threshold can be specified with the option `-threshold_break = (value)`, and finally, the `-save_extrapolative_to = (filename)` option specifies the file for saving the extrapolative (preselected) configurations or non-periodic configuration parts including the extrapolative neighborhoods. If the `-extrapolation_control = true` option is specified but other related options are missed, then the default values will be used.

The control of extrapolation allows for preselecting the configurations (neighborhoods) that are the candidates to be added to the training set. For selecting the configurations (neighborhoods) for which the quantum-mechanical calculations will be conducted, we use the following command

```
mlp select_add pot.almtp train.cfg preselected.cfg  
selected.cfg
```

The `preselected.cfg` file may contain many configurations (neighborhoods), and only a limited number of them will be selected and saved to the `selected.cfg` file. The data for selection (active learning mode and active set) are loaded from the `pot.almtp` file. This command also compares the selected configurations with the ones in the `train.cfg` file; thus, we avoid adding the configurations (and conducting expensive quantum-mechanical calculations) similar to what we already have in the training set.

If we use the “configuration-based” active learning mode, we can directly calculate the periodic configurations from the `selected.cfg` file with any quantum-mechanical package without any additional pre-processing [except for converting to the format of the concrete quantum-mechanical (DFT) package]. However, if we consider the “neighborhoods-based” mode, then the `selected.cfg` file may contain large non-periodic atomistic structures with different extrapolative neighborhoods. Typically, it is not possible to calculate these large atomistic structures with the DFT codes. Therefore, we cut the extrapolative neighborhoods from the large configurations by executing

```
mlp cut_extrapolative_neighborhood selected.cfg  
nbh.cfg -cutoff=8
```

The input file `selected.cfg` contains the column with the grades per atom. This command finds an atom with the largest extrapolation grade and makes the spherical neighborhood `nbh.cfg` including this atom and its neighbors inside the sphere with the given radius `cutoff = 8`.

The list and short descriptions of the mentioned commands can be obtained by executing

```
mlp list  
and  
mlp help <command>
```

The commands `cut_extrapolative_neighborhood` and `convert` are serial, and the rest of the mentioned commands are capable of working with the MPI parallelism.

B. MLIP-3 as a LAMMPS plugin

MTPs implemented in MLIP-3 can be used as interatomic interaction potentials in LAMMPS. To this end, we have to specify the mlip “pair style” of potentials in the LAMMPS input file:

```
pair_style mlip load_from = pot.almtp
pair_coeff * *
```

where the load_from option indicates that the MTP parameters should be loaded from the pot.almtp file. As in the relax command, we can turn on the extrapolation control while running LAMMPS (e.g., molecular dynamics) and preselect extrapolative configurations (neighborhoods):

```
pair_style mlip load_from = pot.almtp
extrapolation_control = true\
threshold_save = 2\
threshold_break = 10\
save_extrapolative_to = preselected.cfg
pair_coeff * *
```

IV. EXAMPLE

Here, we describe how to actively train MTP on atomic neighborhoods starting from one configuration in the training set for the simulation of growing a copper crystal on the (111) surface (in what follows we simply refer to this as “the Cu(111) structure”) by depositing copper atoms on the copper layers (substrate). The files and scripts needed to run this example available at <https://gitlab.com/ivannovikov/mlip-3-example>.

A. System of Cu(111)

Six layers of Cu(111) inside the box of $39 \times 39 \times 40$ Å size (1620 atoms) were taken as a substrate. The distance between the layers was approximately equal to 2.07 Å. We took two top layers (and all the deposited atoms) in the NVE (micro-canonical) ensemble, two middle layers in the NVT (canonical) ensemble (Nose-Hoover thermostat at temperature $T = 670$ K), and finally, two bottom layers were fixed along z axis, i.e., the atoms in these layers were allowed to move only along x and y axes. The substrate and the first deposited atom are shown in Fig. 3.

B. Computational details

We considered the periodic boundary conditions along x and y axes and non-periodic boundary conditions along z axis. We deposited each atom from a height of 40 Å. The MD step was 1 fs. Each atom was deposited in 5000 MD steps “one-by-one” with a speed of 1300 meters per second. In each simulation, we deposited 1380 atoms.

We actively trained MTPs of the 8-th, 12-th, and 16-th levels with 26, 54, and 125 parameters, respectively. We choose $R_{\text{cut}} = 5.0$ Å. We fit MTP energies and forces to the ones calculated with DFT, but we did not fit to DFT stresses as we actively selected and added to the training set only the configurations with non-periodic boundary conditions (i.e., spherical neighborhoods).

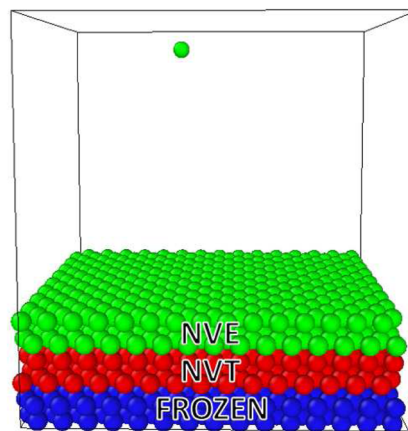


FIG. 3. Scheme of the substrate and the first deposited atom.

We detected neighborhoods (atoms) as extrapolative starting from the threshold $\gamma_{\text{save}} = 2$ (reliable extrapolation, we preselected the neighborhoods with grades greater than this) until the threshold $\gamma_{\text{break}} = 10$ that corresponds to too high (risky) extrapolation. We break the MD simulation once we detected an atom with this grade or higher; next, we selected new neighborhoods among the preselected ones, created the spherical neighborhoods around the atoms with the maximum grades (we took a radius of 8 Å), calculated them with DFT, added them to the training set, and re-trained MTP.

We used the VASP package^{28–31} for DFT calculations. We note that the VASP package operates only with periodic boundary conditions, but we need atoms in a spherical neighborhood to be interacted only inside this neighborhood, not with the ones outside it (i.e., non-periodic boundary conditions). Due to this reason, we added vacuum to the selected spherical neighborhoods along each direction. We found that the optimal size of vacuum along each direction is 9 Å: starting from this size, atoms in different spherical neighborhoods do not interact with each other but interact only inside the same neighborhood. We conducted VASP calculations with the PBE functional³² and the PAW pseudopotentials.³³ We chose the energy cutoff $\text{ENCUT} = 300$ eV and the $1 \times 1 \times 1$ k-mesh.

We note that the details on the convergence of VASP parameters (energy cutoff and k-mesh) and selection of the size of vacuum and the radius of the spherical atomic neighborhood are given in supplementary material.

C. Automatic construction of MTP for simulation of copper layers growing

We first actively trained MTPs of different levels while depositing atoms on the copper substrate. We demonstrate the process of adding configurations to the training set in Fig. 4. For all the potentials, we see that for deposition of the first atom, we need 30 or more DFT calculations. Most DFT calculations were conducted while depositing the first several atoms. Next, this number continues to grow slowly up to depositing approximately the first 200 atoms. We do not need any DFT calculations beyond the deposition of 200 atoms: starting from that point, we grow the Cu(111) structure without any DFT calculations. The number of configurations needed

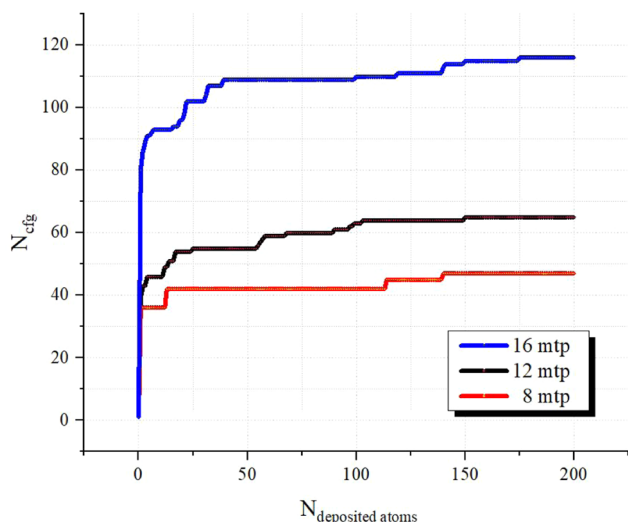


FIG. 4. Amount of DFT calculations (or configurations in the training set) during MD as a function of depositing atoms.

to fit MTP increases with the increase in the level of MTP (or the number of parameters).

In Fig. 5, we demonstrate an example of a large atomic system containing many candidate-extrapolative neighborhoods. In this large system, we find an atom with the maximum extrapolation grade and make a spherical neighborhood around it with the given radius. We further add the neighborhoods like this to the training set.

In Fig. 6, we show examples of neighborhoods added to the training sets during MD simulations. We first add the neighborhood including one copper atom in vacuum as MTP does not “recognize” (or it extrapolates) such a neighborhood. We next add the neighborhood with the same atom, but near the substrate (a). We also add the

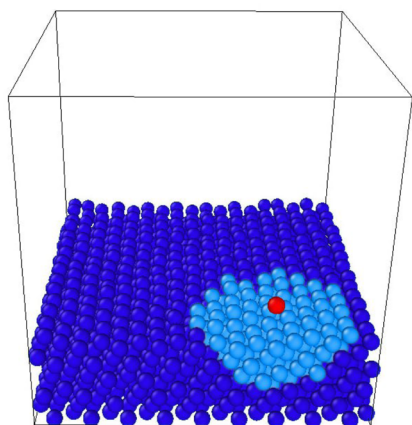


FIG. 5. Large atomic system including candidate extrapolative neighborhoods. The atom with the maximum extrapolation grade and its neighbors are highlighted. These atoms form a spherical neighborhood on which we train the potentials.

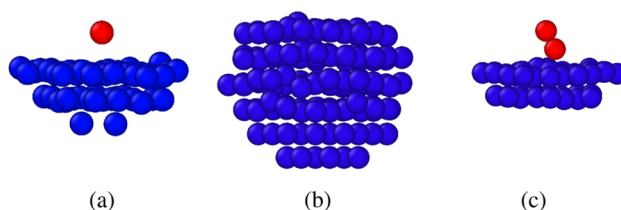


FIG. 6. Examples of atomic neighborhoods selected during active learning. (a) An atom near the substrate, (b) atoms inside the substrate, and (c) several atoms on the substrate. The atoms on the substrate and inside it are of different colors, but all of them are copper.

neighborhood inside the substrate (b). During deposition, the training set saturates, but some neighborhoods with several atoms on the substrate could be still added (c).

The resulting training set sizes for the potential of each level are given in Table I. As was mentioned, the training set size increases with the increase in the level of potential. For estimating the fitting errors and uncertainty of their estimation, we consider an ensemble of five potentials for each level. Namely, in addition to the actively trained potentials of different levels, we passively trained four MTPs. From Table I, we conclude that the fitting errors decrease with the increase in the level of MTPs. We see that the uncertainty of the fitting errors is rather small. As expected, the highest accuracy was reached with the MTP of the highest considered, 16-th, level.

We emphasize that we trained MTPs on the forces on all the atoms and the potential energies of the non-periodic neighborhoods. We investigated one of the neighborhoods and found that the force for the central atom in the neighborhood is close to the one for the same atom in the large configuration from which the neighborhood was cut (see supplementary material). We also analyzed the forces on all the atoms in the training set automatically created for the MTP of 16-th level and found that almost all absolute values of forces are inside the interval (0, 2) eV/Å that is comparable with the forces we have in ordinary periodic configurations (see Fig. 7 with the histogram).

Finally, we demonstrate the Cu<111> structure grown on the Cu<111> substrate with the 16-th level MTP actively trained on atomic neighborhoods in Fig. 8. It could be seen that the grown layers have the same structure and ordering as the substrate. We note that we needed only about a hundred DFT calculations for fitting the

TABLE I. Root mean square fitting errors in energy and forces, and training set size actively selected while training the potentials of levels 8, 12, and 16. The 95% confidence interval is given for the uncertainty of evaluating the fitting errors based on fitting an ensemble of five potentials. The errors decrease with the increase in the level of potentials, and the uncertainty of the error estimation is small.

Level	Training set size	Energy error, meV/atom	Force error, meV/Å
8	48	16.200 ± 0.003	98.878 ± 0.002
12	65	10 ± 1	89 ± 1
16	117	5 ± 2	68 ± 1

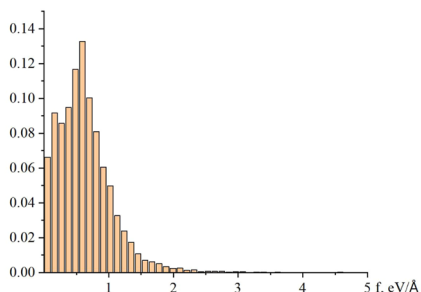


FIG. 7. Absolute forces of all the atoms in the training set constructed for the MTP of the 16-th level.

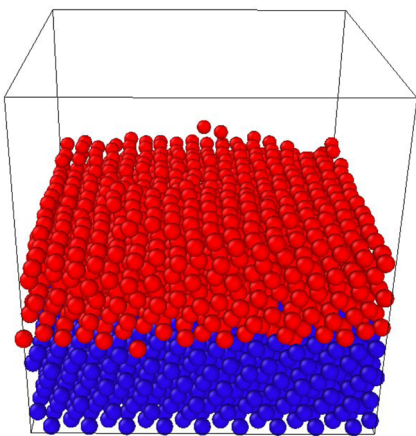


FIG. 8. Cu(111) structure grown on Cu(111) substrate with MTP actively trained on atomic neighborhoods.

MTP, which allows us to simulate the growth of the Cu(111) structure with the algorithm of active learning on atomic neighborhoods described in this paper.

V. CONCLUSIONS

An important part of academic research in computational materials science has been the availability of computer codes enabling atomistic modeling, for example, the free package LAMMPS²⁶ or the commercially available VASP;^{28–31} most of such codes are developed by academic groups. The recent developments in the field of machine-learning potentials necessitate the creation of software packages implementing such potentials. In this paper, we present the MLIP-3 package for constructing and actively training moment tensor potentials. This package not only builds on the MLIP-2 package¹⁶ but also contains a number of improvements, the most prominent being active learning on atomic neighborhoods of a possibly large atomistic simulation. This feature enables performing a large simulation, like deposition of atoms on a surface or a nanoindentation simulation,⁴ and training a potential in a completely automatic and autonomous manner, which leads to the possibility of building such an algorithm into complex computational protocols.^{34,35}

In this manuscript, we described our software package and illustrated its use on a model problem of deposition of Cu atoms on a Cu(111) surface. We hope that our work will be useful for simulating atomic deposition processes, especially for the newly developed materials for which a well-tested interatomic potential does not exist, and hence, the possibility of active learning it while performing the simulation is a very attractive option.

SUPPLEMENTARY MATERIAL

See supplementary material, here we provide the details of the convergence of the potential energy with respect to the VASP parameters (energy cutoff and k-mesh), size of vacuum buffer, and radius of the spherical atomic neighborhood. Additionally, we study the convergence of the force on the central atom in the neighborhood.

ACKNOWLEDGMENTS

This work was supported by the Russian Science Foundation (Grant No. 22-73-10206, <https://rscf.ru/project/22-73-10206/>).

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Evgeny Podryabinkin: Conceptualization (equal); Methodology (equal); Software (equal); Supervision (equal); Validation (equal); Writing – original draft (equal); Writing – review & editing (equal). **Kamil Garifullin:** Visualization (lead); Writing – original draft (equal). **Alexander Shapeev:** Conceptualization (equal); Methodology (equal); Resources (lead); Software (equal); Supervision (equal); Validation (equal); Writing – original draft (equal); Writing – review & editing (equal). **Ivan Novikov:** Project administration (lead); Supervision (equal); Validation (equal); Writing – original draft (equal); Writing – review & editing (equal).

DATA AVAILABILITY

The data that support the findings of this study are available within the article.

REFERENCES

- ¹K. Gubaev, E. V. Podryabinkin, G. L. Hart, and A. V. Shapeev, “Accelerating high-throughput searches for new alloys with active learning of interatomic potentials,” *Comput. Mater. Sci.* **156**, 148–156 (2019).
- ²I. S. Novikov, Y. V. Suleimanov, and A. V. Shapeev, “Automated calculation of thermal rate coefficients using ring polymer molecular dynamics and machine-learning interatomic potentials with active learning,” *Phys. Chem. Chem. Phys.* **20**, 29503–29512 (2018).
- ³B. Mortazavi, E. V. Podryabinkin, I. S. Novikov, T. Rabczuk, X. Zhuang, and A. V. Shapeev, “Accelerating first-principles estimation of thermal conductivity by machine-learning interatomic potentials: A MTP/ShengBTE solution,” *Comput. Phys. Commun.* **258**, 107583 (2021).
- ⁴E. V. Podryabinkin, A. G. Kvashnin, M. Asgarpour, I. I. Maslenikov, D. A. Ovsyannikov, P. B. Sorokin, M. Y. Popov, and A. V. Shapeev, “Nanohardness from

first principles with active learning on atomic environments,” *J. Chem. Theory Comput.* **18**, 1109–1121 (2022).

- ⁵I. I. Novoselov, A. V. Yanilkin, A. V. Shapeev, and E. V. Podryabinkin, “Moment tensor potentials as a promising tool to study diffusion processes,” *Comput. Mater. Sci.* **164**, 46–56 (2019).
- ⁶A. Thompson, L. Swiler, C. Trott, S. Foiles, and G. Tucker, “Spectral neighbor analysis method for automated generation of quantum-accurate interatomic potentials,” *J. Comput. Phys.* **285**, 316–330 (2015).
- ⁷A. P. Bartók, M. C. Payne, R. Kondor, and G. Csányi, “Gaussian approximation potentials: The accuracy of quantum mechanics, without the electrons,” *Phys. Rev. Lett.* **104**, 136403 (2010).
- ⁸J. Behler and M. Parrinello, “Generalized neural-network representation of high-dimensional potential-energy surfaces,” *Phys. Rev. Lett.* **98**, 146401 (2007).
- ⁹A. V. Shapeev, “Moment tensor potentials: A class of systematically improvable interatomic potentials,” *Multiscale Model. Simul.* **14**, 1153–1173 (2016).
- ¹⁰R. Drautz, “Atomic cluster expansion for accurate and transferable interatomic potentials,” *Phys. Rev. B* **99**, 014104 (2019).
- ¹¹G. Dussan, M. Bachmayr, G. Csányi, R. Drautz, S. Etter, C. van der Oord, and C. Ortner, “Atomic cluster expansion: Completeness, efficiency and stability,” *J. Comput. Phys.* **454**, 110946 (2022).
- ¹²H. Wang, L. Zhang, J. Han, and E. Weinan, “DeePMD-kit: A deep learning package for many-body potential energy representation and molecular dynamics,” *Comput. Phys. Commun.* **228**, 178–184 (2018).
- ¹³G. P. Pun, R. Batra, R. Ramprasad, and Y. Mishin, “Physically informed artificial neural networks for atomistic modeling of materials,” *Nat. Commun.* **10**, 2339 (2019).
- ¹⁴S. Batzner, A. Musaelian, L. Sun, M. Geiger, J. P. Mailoa, M. Kornbluth, N. Molinari, T. E. Smidt, and B. Kozinsky, “E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials,” *Nat. Commun.* **13**, 2453 (2022).
- ¹⁵Y. Zuo, C. Chen, X. Li, Z. Deng, Y. Chen, J. Behler, G. Csányi, A. V. Shapeev, A. P. Thompson, M. A. Wood, and S. P. Ong, “Performance and cost assessment of machine learning interatomic potentials,” *J. Phys. Chem. A* **124**, 731–745 (2020).
- ¹⁶I. S. Novikov, K. Gubaev, E. V. Podryabinkin, and A. V. Shapeev, “The MLIP package: Moment tensor potentials with MPI and active learning,” *Mach. Learn.: Sci. Technol.* **2**, 025002 (2021).
- ¹⁷E. V. Podryabinkin, E. V. Tikhonov, A. V. Shapeev, and A. R. Oganov, “Accelerating crystal structure prediction by machine-learning interatomic potentials with active learning,” *Phys. Rev. B* **99**, 064114 (2019).
- ¹⁸I. S. Novikov, A. V. Shapeev, and Y. V. Suleimanov, “Ring polymer molecular dynamics and active learning of moment tensor potential for gas-phase barrierless reactions: Application to $S + H_2$,” *J. Chem. Phys.* **151**, 224105 (2019).
- ¹⁹B. Mortazavi, I. S. Novikov, and A. V. Shapeev, “A machine-learning-based investigation on the mechanical/failure response and thermal conductivity of semiconducting BC_2N monolayers,” *Carbon* **188**, 431–441 (2022).
- ²⁰A. V. Shapeev, E. V. Podryabinkin, K. Gubaev, F. Tasnádi, and I. A. Abrikosov, “Elinvar effect in β -Ti simulated by on-the-fly trained moment tensor potential,” *New J. Phys.* **22**, 113005 (2020).
- ²¹K. Gubaev, E. V. Podryabinkin, and A. V. Shapeev, “Machine learning of molecular properties: Locality and active learning,” *J. Chem. Phys.* **148**, 241727 (2018).
- ²²I. Novikov, B. Grabowski, F. Körmann, and A. Shapeev, “Magnetic moment tensor potentials for collinear spin-polarized materials reproduce different magnetic states of bcc Fe,” *npj Comput. Mater.* **8**, 13 (2022).
- ²³R. Fletcher, “A new variational result for quasi-Newton formulae,” *SIAM J. Optim.* **1**, 18–21 (1991).
- ²⁴E. V. Podryabinkin and A. V. Shapeev, “Active learning of linearly parametrized interatomic potentials,” *Comput. Mater. Sci.* **140**, 171–180 (2017).
- ²⁵S. A. Goreinov, I. V. Oseledets, D. V. Savostyanov, E. E. Tyrtyshnikov, and N. L. Zamarashkin, “How to find a good submatrix,” *Matrix Methods: Theory, Algorithms and Applications: Dedicated to the Memory of Gene Golub* (World Scientific, 2010), pp. 247–256.
- ²⁶S. Plimpton, “Fast parallel algorithms for short-range molecular dynamics,” *J. Comput. Phys.* **117**, 1–19 (1995).
- ²⁷I. S. Novikov, K. Gubaev, E. V. Podryabinkin, and A. V. Shapeev, Supplemental materials: The Mlip Package User Manual, <https://gitlab.com/ashapeev/mlip-2-paper-supp-info/-/blob/master/manual.pdf>.
- ²⁸G. Kresse and J. Hafner, “*Ab initio* molecular dynamics for liquid metals,” *Phys. Rev. B* **47**, 558 (1993).
- ²⁹G. Kresse and J. Hafner, “*Ab initio* molecular-dynamics simulation of the liquid-metal-amorphous-semiconductor transition in germanium,” *Phys. Rev. B* **49**, 14251 (1994).
- ³⁰G. Kresse and J. Furthmüller, “Efficiency of *ab-initio* total energy calculations for metals and semiconductors using a plane-wave basis set,” *Comput. Mater. Sci.* **6**, 15–50 (1996).
- ³¹G. Kresse and J. Furthmüller, “Efficient iterative schemes for *ab initio* total-energy calculations using a plane-wave basis set,” *Phys. Rev. B* **54**, 11169 (1996).
- ³²J. P. Perdew, K. Burke, and M. Ernzerhof, “Generalized gradient approximation made simple,” *Phys. Rev. Lett.* **77**, 3865 (1996).
- ³³P. E. Blöchl, “Projector augmented-wave method,” *Phys. Rev. B* **50**, 17953 (1994).
- ³⁴J. Janssen, S. Surendralal, Y. Lysogorskiy, M. Todorova, T. Hickel, R. Drautz, and J. Neugebauer, “pyiron: An integrated development environment for computational materials science,” *Comput. Mater. Sci.* **163**, 24–36 (2019).
- ³⁵G. Pizzi, A. Cepellotti, R. Sabatini, N. Marzari, and B. Kozinsky, “AiiDA: Automated interactive infrastructure and database for computational science,” *Comput. Mater. Sci.* **111**, 218–230 (2016).